

Having got the access token, you can use it to download data. You need to send two items to the Web server as part of your query. These are the access token and the bounding box. There can be other forms of the API query, but we will focus only on the bounding box. The bounding box is of the format: *lat1, lon1, lat2, lon2*. Here, *lat1* is the latitude and *lon1* is the longitude of the bottom left corner of the bounding box. Similarly, *lat2* and *lon2* are the latitude and longitude of the top right corner of the bounding box.

In this script, I have set the boundary box to cover the extent of India (excluding Andaman and Nicobar Islands). To make the script easier to understand, I have divided the API call URL into two parts (which are both Python strings) — base URL and trailing URL. The trailing URL part contains two parameters — the token and the bounding box. Finally, the two parts are concatenated using the plus (+) operator. One thing to note is that I have used ‘f-strings’, which is a newer way to format strings starting Python 3.6. In Python, f-strings are what may be called ‘formatted string literals’. They begin with an ‘f’ and have curly braces ({ and }), which contain expressions that are replaced by their values at ‘run time’. However, in case you are using an older version of Python, you may have to change this line of code and use something like *str.format()* or % formatting. In case you are interested in knowing more about f-strings, take a look at PEP 498 available at <https://www.python.org/dev/peps/pep-0498/>.

Having constructed the string for the API call, all you need to do is to use it. In this script, I have used a Pandas function *read_json()*. In fact, it's not well known that Pandas can be used to directly download data from a URL. Not only that, the data can be downloaded in a variety of formats. This function *read_json()* returns a JSON object. However, if you are not familiar with the download capabilities of Pandas, you can always use the *popular requests* library as an alternative. In my script I have added some print functions so that you can see the output. Also, the token I have used has certain characters replaced with ‘X’ to preserve privacy:

```
# Author:- Anurag Gupta Email:- 999.anuraggupta@gmail.com
import pandas as pd
import folium
from folium.plugins import HeatMap

###-STEP 1 DOWNLOAD DATA
# See details of API at:- https://aqicn.org/api/
base_url = "https://api.waqi.info"
# Get token from:- https://aqicn.org/data-platform/token/#/
tok = '5c173abcXXXXXX XXXXXXXX482471de46bb352f6'
# (lat, long)-> bottom left, (lat, lon)-> top right
# India is 8N 61E to 37N, 97E approx
latlngbox = "8.0000,61.0000,37.0000,97.0000" # For India
trail_url=f"/map/bounds/?latlng={latlngbox}&token={tok}" #
```

```
my_data = pd.read_json(base_url + trail_url) # Join parts of
URL
print('columns->', my_data.columns) #2 cols 'status' and
'data'
```

The following points about the downloaded data are noteworthy:

- This data has two columns — status and data.
- The status column has the value ‘ok’ for all rows. So, this column is not of much use.
- The data column is in fact a dictionary, which is of our interest. So, we need to convert this column (in the form of a dictionary) to another Pandas DataFrame, which we will do in the next step.

Step 2: Creating a DataFrame table

Even though the downloaded data is a DataFrame, we need to create another DataFrame that has the following four columns: station name, station latitude, station longitude and the air quality index.

To extract these values, we use a *for* loop. The one point to note is that the value corresponding to the key station is itself a dictionary; so in order to extract the station name, we need to use two keys of the type *[each_row['station']]['name']*.

```
###-STEP 2: Create table like DataFrame
all_rows = []
for each_row in my_data['data']:
    all_rows.append([each_row['station']['name'],
                     each_row['lat'],
                     each_row['lon'],
                     each_row['aqi']])

df = pd.DataFrame(all_rows,
                  columns=['station_name', 'lat', 'lon',
                           'aqi'])
```

Step 3: Cleaning the data

The heat map that we intend to create must have a numeric value for the column *aqi*. So we need to convert all values in the column *aqi* to numerics. Moreover, in case a particular value for the column *aqi* cannot be coerced into a numeric, we will replace it with a NaN (Not a Number). To do this, for the DataFrame method *to_numeric()*, we must provide a parameter *errors* with a value of *coerce*. Having done that, we also need to remove all such rows that have a value of NaN for the column *aqi*. Removing NaN can be done by the DataFrame method *df.dropna(subset = ['aqi'])*. The original DataFrame was *df* and its shape was (152, 4), while the new DataFrame (with NaN dropped) is *df1* and its shape is (144, 4). So, we can conclude that there were eight rows in the DataFrame